

פרק 2

מחלקות

מצייני גישה (רמות חשיפה) – Access Modifiers

מצייני גישה לחברי מחלקה (שדות ומתודות) -

חובה לציין גישה עבור כל חבר מחלקה בנפרד
(כמו ב-Java ולהבדיל מ-C++ שם ניתן לציין גישה עבור רשימה של חברי מחלקה)

חמישה מצייני גישה:

- public** – כולם יכולים להשתמש (כמו ב-Java ו-C++)
- private** – רק הקוד של המחלקה יכול להשתמש (כמו ב-Java ו-C++)
- protected** – רק מהקוד של המחלקה או של מחלקות יורשות (כמו ב-C++)
- internal** – רק בקוד שנמצא באותו אסמבלי ניתן לגשת (ברירת המחדל ב-Java)
- protected internal** – איחוד של שניהם (לא חיתוך!) (כמו ב-Java protected)
- ברירת מחדל אם לא רושמים כלום – הנמוך ביותר (private)

מצייני גישה של טיפוסים –

- יש לציין רמת חשיפה גם לטיפוסים (class, enum, struct, etc.)
- public/internal – לטיפוסים לא מקוננים ("חיצוניים")
- טיפוסים מקוננים – כל החמישה אפשריים
- ברירת מחדל אם לא רושמים כלום – הנמוך ביותר (private/internal)

בנאים - Constructors

■ כמו ב- C++ ו- Java:

■ מתודה שאחראית לשלב היצירה של האובייקט

■ שמה כשם המחלקה

■ ללא ערך מוחזר

■ אם לא כתבנו בנאי, קיים בנאי ריק שלא מקבל פרמטרים.

■ אם כתבנו בנאי שמקבל פרמטרים, צריך לכתוב אחד נוסף שלא מקבל פרמטרים במידה ונרצה כזה.

■ כמו ב- Java, ולהבדיל מב- C++:

■ ניתן לאתחל שדות בשורת ההכרזה שלהם (נכנס ב- 11 C++)

■ בנאי אחד יכול להפעיל בנאי אחר

■ אין "שורת איתחול"

```
1. class Student {  
2.     int m_Id;  
3.     string m_Name = string.Empty;  
4.     Course m_Major = new Course("C#");  
  
5.     public Student(int i_ID){  
6.         m_Id = i_ID;  
7.     }  
  
8.     public Student() : this (-1)  
9.     {...}
```

מפרקים - Destructors

- תילדה (~) ושם המחלקה
- מופעל ע"י ה-Garbage Collector בזמנו החופשי
- לרוב לא נממש DTOR
- אם נרצה לשחרר משאבים לא מנוהלים – Dispose
- ב-Java: `public void finalize()`

טיפוסים מקוננים – Nested Types

- ניתן להגדיר מחלקה בתוך מחלקה, enum בתוך מחלקה וכו'
- לטיפוס המקונן יש גישה מלאה לכל השדות של הטיפוס העוטף
- הטיפוס העוטף יכול לגשת רק ל-`public` או `internal` של המקונן
- הטיפוס המקונן מוגדר עם אחד מ-5 מצייני הגישה
- השם של המקונן תמיד כולל את השם של העוטף
- ב-Java: `static nested class` (אין מקבילה ב-C# ל-`Inner class`)

מציינים - Modifiers

static (דומה ל- Java ו- C++)

- שדה/מתודה ברמת המחלקה, לא משוכפל עבור כל אובייקט
- גישה אליו באמצעות שם המחלקה בלבד
- לא ניתן להגדיר משתנה סטטי מקומי (בתוך מתודה)

const (דומה ל- static final בעבודה עם ליטראליים ב- Java ושונה מאוד מ- const של C++)

- הגדרת תווית שמייצגת ערך קבוע שנקבע בזמן קומפילציה
- האתחול מתבצע יחד עם הכרזת התווית
- ניתן להגדיר תווית מקומית (בתוך מתודה)
- התווית מוחלפת בזמן קומפילציה בערך שלה בכל מקום שמשתמשים בה
- גישה באמצעות שם המחלקה בלבד

```
1. class Console {  
2.     private static int    s_WindowHeight;  
3.     private const  int    k_WindowWidth = 130;  
4.     public static void WriteLine(string str){...}
```

מציינים - Modifiers

readonly (דומה ל- final ב- Java) ■

- השדה מקבל את ערכו פעם אחת בזמן יצירת אובייקט
- לא ניתן לשנות את ערכו במהלך חיי האובייקט
- ניתן להציב ערך רק בשורת ההכרזה או בבנאי
- לא ניתן להגדיר משתנה readonly מקומי (בתוך מתודה)

static readonly (דומה ל- static final ב- Java בעבודה עם ערכי זמן ריצה) ■

- שדה המקבל את ערכו פעם אחת בזמן טעינת המחלקה
- לא ניתן לשנות את ערכו בשום שלב אחר
- ניתן להציב ערך בשורת ההכרזה או בבנאי סטטי (יתואר בהמשך)

סוגי בנאים

להבדיל מ- C++, לא קיים "בנאי שיכפול" – Copy Constructor

`Student s2 = s1;`

struct Student (Value Type) – שיכפול רדוד של s1

class Student (Reference Type) - התייחסות נוספת לאובייקט של s1

Clone

private constructor

Factory

Singleton

Service Class

static constructor

static initializer – Java ב-, C++ ב-, לא קיים

מופעל בשלב טעינת המחלקה

לא מקבל פרמטרים

מאפיינים - Properties

זוג פונקציות, get ו-set תחת קורת גג אחת עם חזות של שדה מחלקה

אם רוצים, ניתן לממש רק אחד מהזוג

אם רוצים, ניתן לציין גישה

שונה לכל אחד מהזוג

```
1. class Person
2. {
3.     string m_Name; // private field
4.     int m_Age; // private field
5.
6.     // public properties
7.     public string Name
8.     {
9.         get { return m_Name; }
10.        set { m_Name = value; }
11.    }
12.    public int Age
13.    {
14.        get { return m_Age; }
15.        set { m_Age = value; }
16.    }
17. }
```

```
1. static void Main()
2. {
3.     Person tom = new Person();
4.     // tom.m_Age++; // a compilation error since m_Age is private
5.     tom.Name = "Tom"; // we use the "set" here (line 9)
6.     Console.WriteLine(tom.Name); // we use the "get" here (line 8)
7.     tom.Age = tom.Age + 1; // using both the "setter" and "getter" here
8.     tom.Age++; // using both the "setter" and "getter" here (lines 14,15)
9. }
```


Indexers

דומה להעמסת אופרטור [] ב- C++

ניתן לקבל אינדקס מספרי

ניתן לקבל string

ניתן לקבל יותר מפרמטר אחד

```
1. class Student
2. {
3.     private List<float> m_Grades;
4.
5.     // public indexer
6.     public float this[int i_Idx]
7.     {
8.         get { return m_Grades[i_Idx]; }
9.         set { m_Grades[i_Idx] = value; }
10.    }
```

```
1. static void Main()
2. {
3.     Student tom = new Student();
4.     // tom.m_Grades.Clear(); // a compilation error since m_Grades is private
5.     tom[4] = 94; // we use the "set" here (line 8)
6.     Console.WriteLine(tom[4]); // we use the "get" here (line 7)
7. }
```